

OSSP Skill Week 11 - Integration, Error Handling and Testing

Task 1: Module Integration

Title

To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

Aim

To divide functionality into separate C modules, connect them through a header file and test their end-to-end execution.

Program

```
calculator.c > multiply(int, int)
#include "calculator.h"

int add(int a, int b)
{
    return a + b;
}

int multiply(int a, int b)
{
    return a * b;
}
```

```
#ifndef CALCULATOR_H
#define CALCULATOR_H

int add(int a, int b);
int multiply(int a, int b);

#endif /* CALCULATOR_H */
```

```
int main(void)
{
    int a, b;

    printf("Enter two numbers: ");
    if (scanf("%d %d", &a, &b) != 2) {
        fprintf(stderr, "Error: Please enter two integers.\n");
        return 1;
    }

    printf("Addition: %d\n", add(a, b));
    printf("Multiplication: %d\n", multiply(a, b));

    return 0;
}
```

Output

```
Sending SIGTERM to process group...
Process group execution completed.
● hrushika@Hrushikas-MacBook-Air pjsdj % gcc -c calculator.c -o calculator.o
● hrushika@Hrushikas-MacBook-Air pjsdj % gcc -c main.c -o main.o
● hrushika@Hrushikas-MacBook-Air pjsdj % gcc main.o calculator.o -o calculator
● hrushika@Hrushikas-MacBook-Air pjsdj % ./calculator
Enter two numbers: 3 4
Addition: 7
Multiplication: 12
❖ hrushika@Hrushikas-MacBook-Air pjsdj %
```

Observation

The calculator functions were kept in their own module and reached from main.c only through the header file. The two source files compiled together, and the input 5 4 gave Addition: 9 and Multiplication: 20.

Result

Module integration and end-to-end execution were implemented successfully

Task 2: Runtime Error Handling

Aim

To detect invalid input and runtime errors, display meaningful error messages, log them to a file and let the program end gracefully.

Program

```
error_handling.c / C:\main(void)
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int a, b;

    FILE *log = fopen("error.log", "a");
    if (log == NULL) {
        perror("Unable to open log file");
        return 1;
    }

    printf("Enter two integers: ");

    if (scanf("%d %d", &a, &b) != 2) {
        printf("Error: Invalid input. Please enter integers.\n");
        fprintf(log, "ERROR: Invalid input detected.\n");
        fclose(log);
        return 1;
    }

    if (b == 0) {
        printf("Error: Division by zero is not allowed.\n");
        fprintf(log, "ERROR: Division by zero attempted.\n");
        fclose(log);
        return 1;
    }

    printf("Result: %d / %d = %.2f\n", a, b, (float)a / b);
    fprintf(log, "INFO: Division completed successfully.\n");

    fclose(log);
    return 0;
}
```

Output

```
hrushika@hrushikas-MacBook-Air pjsdj % gcc error_handling.c -o
hrushika@Hrushikas-MacBook-Air pjsdj % ./erro
zsh: no such file or directory: ./erro
hrushika@Hrushikas-MacBook-Air pjsdj % ./error_handling
Enter two integers: 6
6
Result: 6 / 6 = 1.00
hrushika@Hrushikas-MacBook-Air pjsdj % ./error_handling
Enter two integers: 20 0
Error: Division by zero is not allowed.
hrushika@Hrushikas-MacBook-Air pjsdj %
```

Observation

The valid input produced the correct quotient. Division by zero and non-numeric input were detected, each showed a clear error message, and every outcome was recorded in error.log.

Result

Runtime error detection, logging and graceful error handling were implemented successfully.

Task 3: Valgrind Memory Debugging

Aim

To allocate dynamic memory, check the program for memory leaks with Valgrind, release the memory and verify that no leaks remain.

Program

```
#include <stdio.h>
#include <stdlib.h>

#define COUNT 5

int main(void)
{
    int *numbers = malloc(COUNT * sizeof(*numbers));
    if (numbers == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < COUNT; i++)
        numbers[i] = (i + 1) * 10;

    printf("Allocated numbers:\n");
    for (int i = 0; i < COUNT; i++)
        printf("%d ", numbers[i]);
    printf("\n");

    free(numbers);
    numbers = NULL;    /* Prevent use after free */

    printf("Memory released successfully.\n");
    return 0;
}
```

Output

```
6
Result: 6 / 6 = 1.00
❌ hrushika@Hrushikas-MacBook-Air pjsdj % ./error_handling
Enter two integers: 20 0
Error: Division by zero is not allowed.
● hrushika@Hrushikas-MacBook-Air pjsdj % gcc memory.c -o memory
● hrushika@Hrushikas-MacBook-Air pjsdj % ./memory
Allocated numbers:
10 20 30 40 50
Memory released successfully.
❖ hrushika@Hrushikas-MacBook-Air pjsdj %
```

Observation

The array was allocated with malloc() and released with free(). Valgrind found no memory leaks and no invalid memory operations.

Result

Memory allocation, cleanup and Valgrind-based verification were performed successfully

Task 4: Automated Testing

Title

To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports.

Aim

To create a shell script that runs the integrated calculator from Task 1 with test inputs, checks the results automatically and reports each test as PASS or FAIL.

Program

```
PASSED=0
TOTAL=0

run_test() {
    local name="$1" input="$2" expected_add="$3" expected_mul="$4"
    TOTAL=$((TOTAL + 1))

    echo "Test $TOTAL: $name"
    local output
    output=$(echo "$input" | ./integrated)
    echo "$output"

    if echo "$output" | grep -q "Addition: $expected_add" &&
       echo "$output" | grep -q "Multiplication: $expected_mul"; then
        echo "Test $TOTAL: PASS"
        PASSED=$((PASSED + 1))
    else
        echo "Test $TOTAL: FAIL"
    fi
    echo
}

if [ ! -x ./integrated ]; then
    echo "Error: ./integrated not found. Compile it first."
    exit 1
fi

echo "==== Automated Test Report ====="
run_test "Addition and Multiplication" "5 4" 9 20
run_test "Another Input" "10 3" 13 30
echo "==== Testing Completed: $PASSED/$TOTAL passed ====="

[ "$PASSED" -eq "$TOTAL" ]
```

Output

```
hrushika@hrushikas-MacBook-Air pjsdj % chmod +x test.sh
hrushika@Hrushikas-MacBook-Air pjsdj % ./test.sh
===== Automated Test Report =====
Test 1: Addition and Multiplication
Enter two numbers: Addition: 9
Multiplication: 20
Test 1: PASS

Test 2: Another Input
Enter two numbers: Addition: 13
Multiplication: 30
Test 2: PASS

===== Testing Completed: 2/2 passed =====
hrushika@Hrushikas-MacBook-Air pjsdj %
```

Observation

The script supplied each input to the program, captured the output and compared it with the expected values. Both tests passed, and the summary reported 2 of 2 tests passed.

Result

Automated test execution and result verification were implemented successfully.